

AGOOJIYE HMI

Documentation logicielle

Interface embarquée de la navette électrique

Documentation complète de la partie logicielle de la navette électrique
AGOOJIYE : ce qui est utilisé, comment on lance, comment ça fonctionne,
et sur quoi ça tourne.

Trois documents se partagent le sujet, sans se répéter :

DOCUMENT	RÉPOND À
README.md	Je viens de cloner, je fais quoi ? — démarrage en trois commandes
DOCUMENTATION.md (ce fichier)	Comment c'est fait, et pourquoi ? — technologies, architecture, exécution
BACKEND.md	Comment je branche le vrai véhicule ? — inventaire des propriétés, trois méthodes de branchement

Sommaire

1 Ce que c'est

Ce que le logiciel garantit

2 Sur quoi ça tourne

Cible embarquée

Poste de développement

3 Les technologies, et pourquoi celles-là

Ce qui n'est **pas** utilisé, volontairement

4 Installer, construire, lancer

4.1 Dépendances

4.2 Lancer

4.3 Options de `run.sh`

4.4 Rejouer un scénario

4.5 Construire à la main

4.6 Ce que produit la construction

5 Structure du dépôt

6 Comment ça fonctionne

6.1 Trois couches, une seule direction

6.2 Le démarrage, et le verrou haute tension

6.3 La navigation

6.4 Les alertes et la qualité des signaux

6.5 La simulation

6.6 Le rendu : pourquoi tout est dessiné à la main

6.7 L'assistant vocal

7 Les écrans

8 Tester

8.1 Le test de fumée

8.2 Relever l'état véhicule en CSV

8.3 Variables d'environnement de recette

8.4 Ce qui n'est pas encore mesuré

9 Passer en production

10 Dépannage

11 Pour aller plus loin

1. Ce que c'est

Une interface homme-machine (IHM) embarquée pour une **navette électrique ouverte** : le tableau de bord que le conducteur a sous les yeux. Elle affiche la vitesse, l'énergie, l'état des organes, la navigation, le média, les aides à la conduite, et elle contrôle la chaîne haute tension au démarrage.

Ce n'est **pas une application web déguisée**. C'est un binaire natif qui s'ouvre en plein écran, sans navigateur, sans serveur, sans réseau, et qui doit afficher une image correcte à la seconde où le véhicule est mis sous tension.

Ce que le logiciel garantit

Trois propriétés sont tenues par la structure du code, pas par la vigilance du développeur :

1. **Une valeur affichée vient toujours d'une seule source.** Aucun écran ne calcule une donnée métier.
2. **Une donnée absente s'affiche - - , jamais une valeur inventée.** Un capteur muet lève une alerte de lui-même.
3. **Un défaut d'isolement haute tension interdit le démarrage**, et ce refus ne se contourne par aucun chemin de l'interface.

2. Sur quoi ça tourne

Cible embarquée

Système	Linux embarqué (Yocto, Buildroot, Debian embarqué...)
Processeur	ARM 64 bits ou x86-64
Affichage	1600 × 900 logique, plein écran
Accélération graphique	non requise
Réseau	non requis au fonctionnement
Audio	facultatif (assistant vocal)

La ligne la plus importante de ce tableau est « **accélération graphique : non requise** ». La cible n'a pas forcément de GPU utilisable, et le rendu peut tomber en logiciel (`llvmpipe`, `QT_QUICK_BACKEND=software`). Toute la conception graphique découle de cette contrainte — voir §6.6.

Plateformes Qt utilisables selon l'intégration : `eglfs` (plein écran direct sur le framebuffer, le cas embarqué classique), `wayland`, `xcb`, ou `offscreen` pour la recette automatisée.

Poste de développement

Système	Linux (Debian, Ubuntu, Kali testés)
Compilateur	GCC ou Clang, C++17
CMake	≥ 3.16
Qt	Qt 6 (6.5 ou plus récent)

macOS et Windows compilent en principe — rien dans le code n'est spécifique à Linux — mais seule la ligne Linux est utilisée et vérifiée. Le repli vocal `spd-say`, lui, est spécifique à Linux.

3. Les technologies, et pourquoi celles-là

Technologie	Rôle	Pourquoi elle
Qt 6 / QML	Toute l'interface	Standard de fait de l'IHM embarquée automobile. Déclaratif, animations intégrées, empreinte maîtrisée.
Qt Quick	Moteur de rendu	Rendu scène-graphe, capable de tomber en logiciel sans changer une ligne de code.
C++17	Point d'entrée, polices, outillage de recette	Le minimum : Qt charge le QML, et le C++ n'intervient que là où QML ne peut pas.
CMake	Construction	<code>qt_add_qml_module</code> compile le QML et les ressources dans le binaire.
Canvas 2D (QML)	Halos, arc de batterie, vignette	Dessin à la main, sans shader — le seul moyen d'obtenir la même image avec ou sans GPU.
Phosphor Icons (police)	Toutes les icônes	Une police, pas des images : n'importe quelle taille reste nette, et le jeu complet pèse 3 Mo.
Inter (police)	Tout le texte	Dessinée pour les écrans, lisible aux petites tailles et de loin.
Qt TextToSpeech	Assistant vocal	Chemin propre quand le module et son plugin sont installés.
<code>spd-say</code> (speech-dispatcher)	Repli vocal	Existe parce que le plugin Qt manque sur beaucoup d'installations.
Bash	<code>run.sh</code> , <code>test.sh</code>	Aucune dépendance à installer pour construire, lancer et tester.

Ce qui n'est pas utilisé, volontairement

- **Aucun `ShaderEffect`, aucun `ShaderEffectSource`**. Ils disparaissent sans message d'erreur en rendu logiciel — l'écran serait juste vide sur la cible.
- **Aucune bibliothèque tierce**, aucun gestionnaire de paquets (npm, conan, vcpkg). Le dépôt cloné se construit avec Qt et un compilateur, rien d'autre.
- **Aucun accès réseau** au fonctionnement. Les cartes et pochettes d'album sont des images embarquées.
- **Aucune base de données**. L'état vit en mémoire, la persistance viendra du véhicule.

4. Installer, construire, lancer

4.1 Dépendances

Debian / Ubuntu / Kali :

```
sudo apt install build-essential cmake \  
qt6-base-dev qt6-declarative-dev \  
qml6-module-qtquick qml6-module-qtquick-controls \  
qml6-module-qtquick-shapes qml6-module-qtquick-window
```

Pour l'assistant vocal (facultatif) :

```
sudo apt install speech-dispatcher      # fournit spd-say – suffit  
sudo apt install qt6-speech-speechd-plugin # chemin Qt natif, si disponible
```

Vérification : si `spd-say -l fr "Bienvenue"` parle dans un terminal, l'assistant parlera dans l'application.

4.2 Lancer

```
git clone git@github.com:AGBECIDavid/maquette.git  
cd maquette/agoojiye-hmi  
./run.sh
```

`run.sh` configure, construit, puis lance. Il nomme toute dépendance manquante avec la commande qui l'installe.

Si Qt vient de l'installateur officiel plutôt que du système :

```
CMAKE_PREFIX_PATH=$HOME/Qt/6.8.3/gcc_64 ./run.sh
```

4.3 Options de `run.sh`

```
./run.sh --build    # construit seulement, ne lance pas  
./run.sh --clean    # repart d'un dossier de build vide  
./run.sh --help     # liste tout
```

4.4 Rejouer un scénario

Le scénario se passe **en argument**, pas en variable d'environnement : il n'y a aucun nom à retenir.

```
./run.sh 00000      # chaîne haute tension saine (défaut)  
./run.sh 01000      # batterie de traction en défaut → écran rouge, contournable  
./run.sh 10000      # isolement châssis en défaut → démarrage refusé  
./run.sh 11111      # toute la chaîne au tapis  
  
./run.sh tyre       # pression des pneus  
./run.sh belt       # ceinture non bouclée  
./run.sh battery     # batterie critique  
./run.sh sensor      # capteur de vitesse muet  
./run.sh fault       # défaut système  
  
./run.sh 01000 tyre  # les deux se combinent
```

Un argument mal tapé est **refusé** avec la liste des valeurs valides, plutôt qu'absorbé en silence : au niveau du shell, une faute de frappe est délibérée.

4.5 Construire à la main

`run.sh` n'est qu'une commodité. Le projet est un projet CMake ordinaire :

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
./build/agoojiye-hmi
```

4.6 Ce que produit la construction

Un seul fichier exécutable, `build/agoojiye-hmi`. Le QML, les polices et les images sont compilés *dedans* par `qt_add_qml_module` : il n'y a pas de dossier de ressources à déployer à côté, pas de chemin relatif à respecter. Copier le binaire sur la cible suffit — avec les bibliothèques Qt, qui restent des dépendances partagées.

Détails de construction qui comptent :

- **Les singletons QML** (`Theme`, `AppState`, `VehicleData`, `VehicleSimulator`) sont déclarés par `set_source_files_properties(... QT_QML_SINGLETON_TYPE TRUE)`, **avant** `qt_add_qml_module`. Après, la propriété est ignorée sans erreur et l'application échoue au chargement.
- **Le préfixe de ressources est** `qrc:/AgoojiyeHMI/`, fixé par `URI AgoojiyeHMI`.
- **Qt TextToSpeech est cherché en** `QUIET` : absent, la construction réussit à l'identique et `AGOOJIYE_HAS_TTS` n'est pas défini.

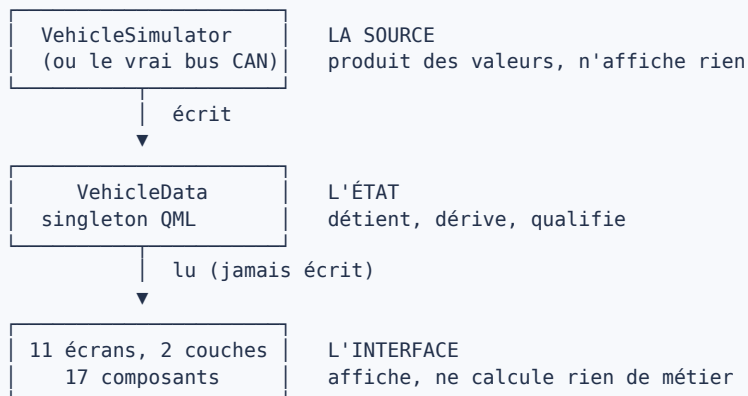
5. Structure du dépôt

```
agoojiye-hmi/
├─ CMakeLists.txt      construction, ressources, singletons
├─ main.cpp            point d'entrée + outillage de recette (208 l.)
├─ voiceannouncer.{h,cpp} assistant vocal, deux chemins + repli
├─ run.sh              construire et lancer, scénarios en argument
├─ test.sh             test de fumée en 8 étapes, sans écran
├─ qml/
│   ├── Main.qml       coque : barres, écrans, couches (141 l.)
│   ├── Theme.qml      couleurs et polices, singleton (49 l.)
│   ├── AppState.qml   état d'interface, singleton (204 l.)
│   ├── VehicleData.qml ← POINT DE BRANCHEMENT DU BACKEND (357 l.)
│   ├── VehicleSimulator.qml source simulée, singleton (309 l.)
│   ├── VoiceCommands.qml commandes vocales, singleton (412 l.)
│   ├── Icons.js       table des points de code Phosphor
│   ├── components/    18 briques réutilisées (1 587 l.)
│   └─ screens/        11 écrans + 2 couches (4 910 l.)
└─ assets/
    ├── fonts/         Phosphor (2) + Inter (4) – 3,3 Mo
    └─ images/         rendus du véhicule, cartes, pochettes – 1,8 Mo
```

Environ **8 600 lignes** de QML, C++ et JavaScript.

6. Comment ça fonctionne

6.1 Trois couches, une seule direction



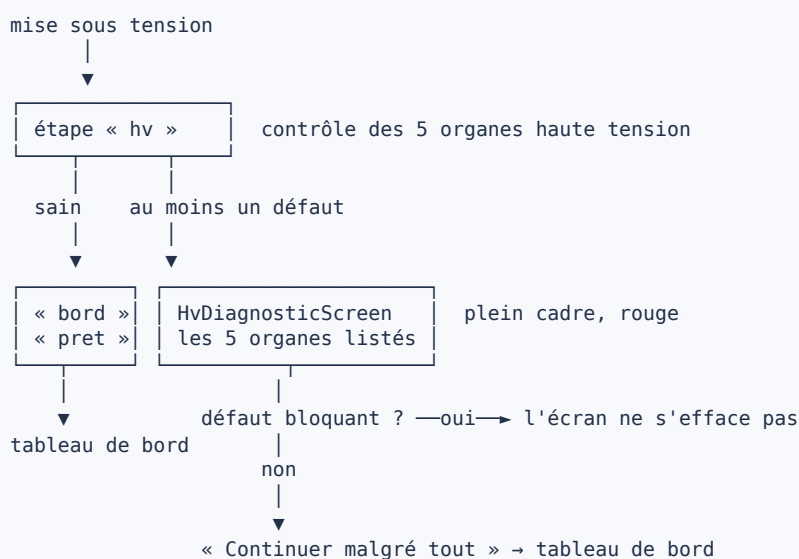
La règle tient en une phrase : aucune valeur métier n'est écrite dans un écran. C'est ce qui rend le branchement du vrai véhicule indolore — on coupe le simulateur (`running: false`), on alimente `VehicleData`, et aucun écran ne change. Les trois méthodes de branchement sont dans [BACKEND.md](#).

C'est aussi ce qui permet de corriger une incohérence **à la source** plutôt qu'écran par écran : « rouler avec le frein de stationnement serré » n'est pas corrigé quelque part, c'est **irreprésentable**, parce qu'un seul fichier décide des deux valeurs.

`AppState` est la quatrième pièce, à côté et non dedans : il porte l'état **d'interface** (quel écran est ouvert, quelles options l'utilisateur a basculées) et rien du véhicule.

6.2 Le démarrage, et le verrou haute tension

La séquence de démarrage est une **couche**, pas un écran. Elle n'a pas de clé de navigation, donc aucun bouton ne peut la rejouer — c'est une propriété structurelle, pas une convention.



Les cinq organes contrôlés, dans l'ordre du motif :

Position	Organe	Bloquant
1	NON-Contact Châssis — Circuit HT (isolement)	oui
2	Batterie de traction	non
3	BMS	non
4	OBC	non
5	Moteur	non

0 = conforme, 1 = en défaut. Un défaut d'isolement bloque : un circuit haute tension en contact avec la caisse met les passagers sous tension, et aucun confort d'usage ne justifie de passer outre.

Quatre détails de sûreté valent d'être connus :

- Le bouton « **Passer** » saute la mise en scène, **pas le contrôle**. S'il reste un défaut, il conduit au diagnostic. Le lien est d'ailleurs masqué pendant l'étape haute tension.
- Le refus est vérifié dans `AppState.dismissHvDiagnostic()`, pas dans l'écran — pour qu'aucun autre appelant ne puisse le contourner.
- L'écran de diagnostic montre **les cinq organes**, pas seulement ceux en défaut : savoir ce qui a été contrôlé et jugé sain fait partie du diagnostic.
- « Relancer le diagnostic » **réinterroge** la chaîne, il n'efface rien.

6.3 La navigation

Sept destinations permanentes dans la barre du bas, décrites une seule fois dans `AppState.navSections` :

#	Destination	Clé	Écrans rattachés
1	Accueil	<code>dash</code>	<code>dash</code>
2	Navigation	<code>nav</code>	<code>nav</code>
3	Véhicule	<code>veh</code>	<code>veh</code> , <code>entretien</code>
4	Conduite	<code>conduite</code>	<code>conduite</code>
5	ADAS	<code>adas</code>	<code>adas</code>
6	Média	<code>media</code>	<code>media</code> , <code>mediaNow</code>
7	Paramètres	<code>parametres</code>	<code>parametres</code>

Les principes tenus :

- **Toute destination est à un seul appui**, depuis n'importe où. On ne repasse jamais par l'accueil.
- **Un écran secondaire garde allumé le bouton de sa section**. Le lecteur média allume « Média » ; l'entretien allume « Véhicule ». L'utilisateur sait toujours où il est.
- `AppState.go()` **est le seul point d'entrée**. Il ne connaît que des écrans, donc la séquence de démarrage lui est inaccessible par construction.
- **Les sections des barres latérales sont mémorisées**. Partir ailleurs et revenir retombe sur la section qu'on avait ouverte, pas sur la première.
- Le menu « toutes les applications » n'occupe pas un des sept boutons : il s'ouvre depuis la barre du haut, comme un tiroir.

6.4 Les alertes et la qualité des signaux

Deux mécanismes qu'un tableau de bord doit avoir, et qui sont souvent absents d'une maquette.

Qualité du signal. Chaque mesure porte un état : **OK**, **STALE** (périmée) ou **MISSING** (absente).

```
VehicleData.valid("speed")           // false si le capteur est muet
VehicleData.reading("speed", v, 0)    // "48" si valide, "- -" sinon
```

Une valeur absente s'affiche **- -**. Elle n'est jamais remplacée par la dernière valeur connue, ni par zéro : un conducteur qui lit **0 km/h** croit être à l'arrêt.

Alertes dérivées. `VehicleData.activeAlerts` est **calculé**, jamais poussé : il n'existe aucun moyen de lever une alerte qui ne corresponde pas à un état réel, ni d'oublier de l'éteindre. Trois niveaux — **CRITICAL**, **WARNING**, **INFO** — et un ordre d'émission fixe, qui décide aussi quelle alerte le bandeau retient (`topAlert` = la première) : frein de stationnement, ceinture, haute tension, défaut système, batterie, pneus, dépassement de vitesse, signaux manquants, ouvrants.

Le bandeau `AlertBanner` **prend sa place dans la colonne** au lieu de se poser par-dessus l'écran : recouvrir une commande au moment précis où le conducteur la cherchait est pire que de décaler l'affichage de quelques dizaines de pixels.

6.5 La simulation

`VehicleSimulator` occupe exactement la place du futur lien véhicule. Il ne fait pas bouger des chiffres au hasard — une valeur qui saute trahit la maquette immédiatement.

Le modèle respecte :

Grandeur	Valeur	Effet
Accélération max	1,1 m/s ²	la vitesse ne se téléporte jamais
Freinage max	1,8 m/s ²	freinage confortable, pas un mur
Masse	1 450 kg	bilan d'énergie crédible
Résistance au roulement	0,014	—
Trainée	2,6 m ² × Cx	carrosserie ouverte
Rendement traction	0,86	—
Rendement régénération	0,55	—
Charges auxiliaires	0,8 kW	écran, éclairage, pompes

Le scénario joué est celui d'une navette en service : **arrêt** → **roulage** → **ralentissement** → **arrêt**, en boucle.

Le pas d'intégration est le temps réellement écoulé, pas la cadence visée. Un minuteur Qt n'est pas ponctuel : sous charge, en rendu logiciel ou après un ramasse-miettes, il arrive en retard. Supposer 100 ms fixes ferait dériver le véhicule simulé du monde réel — d'autant plus que la cible est modeste. Le pas est plafonné à 0,35 s : après une longue interruption on ne rattrape pas d'un bond.

Un mode `deterministic` fixe les durées de phase, pour que la recette mesure le code et non un tirage au sort.

6.6 Le rendu : pourquoi tout est dessiné à la main

`Glow`, `BatteryArc`, `Vignette` sont peints en **Canvas 2D**, et les fonds sont des dégradés QML. Il n'y a pas un seul shader dans le projet.

C'est délibéré. Un `ShaderEffect` sur une cible sans accélération matérielle ne produit pas une erreur : il **disparaît en silence**. Un halo absent passe inaperçu en revue et saute aux yeux dans le véhicule. En dessinant à la main, le rendu est identique

partout — simplement plus ou moins rapide.

Le thème est centralisé dans `Theme.qml` : bleu de marque `#3b82f6` sur fond navy très sombre, vert pour ce qui va bien, orange et rouge pour ce qui ne va pas. `HvDiagnosticScreen` est le seul écran qui n'utilise pas le bleu de marque — le rouge y est la couleur porteuse, parce qu'il ne dit pas autre chose.

6.7 L'assistant vocal

Deux moitiés, et une seule est encore à faire.

Ce qu'il comprend — fait. `VoiceCommands.qml` porte une table de 32 commandes, ses règles de refus et de confirmation, et la grammaire qu'un moteur de reconnaissance recevra. Tout fonctionne dès aujourd'hui, sans micro, depuis *Paramètres* → *Assistant vocal*.

Ce qu'il entend — à brancher. Un moteur de reconnaissance local (Vosk en tête de liste : modèles d'environ 50 Mo, grammaire restreinte, API C) n'aura qu'à transmettre le texte reconnu à `VoiceCommands.handle()`.

Les choix qui tiennent l'ensemble :

- **Vocabulaire fermé.** Le moteur ne peut rendre que des phrases de la table : une commande inventée par une erreur de reconnaissance devient impossible plutôt que rare.
- **La voix est un périphérique d'entrée, pas une couche.** Chaque commande appelle les fonctions que le tactile appelle déjà ; les règles de sécurité n'existent qu'en un exemplaire.
- **Ce qui ne doit jamais se commander à la voix est absent de la table**, pas désactivé : conduite, frein de stationnement, chaîne haute tension, régulateur, freinage d'urgence.
- **L'autorisation se revérifie au moment d'agir.** Un « oui » prononcé après que la navette s'est mise à rouler ne rouvre rien.

Détail des niveaux, ajout d'une commande et branchement d'un moteur : [BACKEND.md](#).

Le vrai risque est matériel : la navette est ouverte, donc exposée au vent et au bruit de roulement. Un bouton d'appui-pour-parler au volant sera plus fiable qu'un mot de réveil, et un micro directionnel orienté vers le conducteur comptera plus que le choix du modèle.

Pour la sortie, deux chemins, essayés dans cet ordre :

1. **Qt TextToSpeech**, quand le module et son plugin de sortie (`libqtexttospeech_speechd.so`) sont installés.
2. `spd-say -w -l fr -r -15` via `QProcess`, le client en ligne de commande de speech-dispatcher.

Le second existe parce que le premier demande un plugin qui manque sur beaucoup d'installations, alors que speech-dispatcher lui-même est souvent déjà là. C'est la cause du symptôme classique : `spd-say` parle dans le terminal mais l'application reste muette — le module Qt est trouvé, son plugin de sortie ne l'est pas.

Sans aucun des deux, l'application compile et tourne à l'identique : `speak()` ne fait rien, `available` reste faux, et l'interface s'appuie sur son affichage (bulle + onde animée) pour rester lisible. Le destructeur tue le processus et appelle `spd-say -C` : fermer l'application coupe la voix.

7. Les écrans

11 écrans et 2 couches, 39 panneaux atteignables au total (un écran à barre latérale compte une entrée par section).

Écran	Clé	Sections de la barre latérale
Tableau de bord	<code>dash</code>	— (écran de référence : vitesse, énergie, alertes)
Navigation	<code>nav</code>	— (carte, itinéraire, prochaine manœuvre)
Véhicule	<code>veh</code>	Aperçu · Ouvrants et accès · Pression des pneus · Énergie · Températures · Informations
Entretien	<code>entretien</code>	— (sous-écran de Véhicule)
Conduite	<code>conduite</code>	Modes de conduite · Régénération · Traction · Direction · Suspension · Freinage
ADAS	<code>adas</code>	Aides à la conduite · Régulateur de vitesse · Sécurité · Stationnement · Vision · Alerte conducteur
Média	<code>media</code>	Musique · Radio · Bluetooth · USB · Apple CarPlay · Android Auto
Lecture en cours	<code>mediaNow</code>	Lecture · Playlists · Sources · Paramètres audio
Paramètres	<code>parametres</code>	Général · Affichage · Son · Véhicule · Système · Assistant vocal
Téléphone	<code>phone</code>	—
Menu	<code>menu</code>	— (tiroir « toutes les applications »)
Démarrage	<i>(aucune)</i>	couche, pas écran
Diagnostic HT	<i>(aucune)</i>	couche, pas écran

Les modes de conduite sont ECO, NORMAL, SPORT — poussés vers `VehicleData` et non gardés dans l'interface, pour que le backend les voie.

8. Tester

8.1 Le test de fumée

```
./test.sh
```

Pas besoin d'écran : Qt tourne en mode *offscreen*. À lancer avant chaque `git push`.

Huit étapes :

#	Étape	Échoue si
1	Construction	la compilation échoue
2	Parcours de l'interface	Qt émet le moindre avertissement QML
3	Présence des panneaux	un des 39 panneaux ne produit pas de capture
4	Séquence de démarrage	la séquence ne se joue pas jusqu'au bout
5	Cohérence physique sur 30 s	frein de stationnement en roulant, rapport P en roulant, autonomie négative, accélération hors bornes, ou les 3 phases pas toutes vues
6	Chaîne d'alerte	une panne injectée ne remonte pas au bandeau
7	Verrou haute tension	un motif est mal rapporté, ou l'isolement n'est pas bloquant
8	Assistant vocal	une commande hors table est acceptée, un ouvrant s'ouvre en roulant, un « oui » agit après que la navette a démarré, ou un capteur muet est annoncé comme une valeur

L'étape 2 est celle qui attrape les vraies régressions : **une dépendance circulaire ou une propriété inconnue ne casse pas la compilation**, elle fait s'effondrer une mise en page en silence.

Pour regarder le résultat :

```
./test.sh --keep /tmp/captures
```

8.2 Relever l'état véhicule en CSV

```
HMI_TRACE=30 ./build/agoojiye-hmi > trace.csv
```

Une ligne toutes les 200 ms, 18 colonnes, puis l'application se ferme. Cet outil existe pour que la simulation se vérifie **comme une donnée** plutôt qu'en regardant l'écran : « le frein de stationnement n'est jamais serré en roulant » est une comparaison de colonnes ici, et une opinion autrement.

La colonne `wall` porte le temps réellement écoulé. Sans elle, une accélération se déduirait de l'intervalle *visé* entre deux relevés — or le relevé peut lui aussi arriver en retard, et le contrôle accuserait la physique d'un défaut qui n'appartient qu'à l'échantillonnage.

8.3 Variables d'environnement de recette

Variable	Effet
<code>HMI_HV=<5 chiffres></code>	applique un motif haute tension
<code>HMI_FAULT=<mot></code>	injecte une panne : <code>belt</code> , <code>tyre</code> , <code>battery</code> , <code>sensor</code> , <code>fault</code>
<code>HMI_TRACE=<s></code>	relevé CSV pendant N secondes, puis quitte (force le mode déterministe)
<code>HMI_SCREENSHOT_DIR=<dir></code>	parcourt les 39 panneaux, une capture chacun, puis quitte
<code>HMI_VOICE="a b ..."</code>	rejoue des phrases comme si elles avaient été reconnues ; <code>@vitesse=N</code> change la vitesse entre deux
<code>HMI_BOOT_FRAMES=<dir></code>	capture la séquence de démarrage image par image

Les arguments de `run.sh` ne font que poser `HMI_HV` et `HMI_FAULT` pour vous.

8.4 Ce qui n'est pas encore mesuré

La fluidité d'animation sur le matériel réel n'a jamais été mesurée. Toutes les exécutions de recette passent par `QT_QPA_PLATFORM=offscreen`, qui ne dit rien du nombre d'images par seconde sur la cible. C'est la première chose à faire dès qu'un exemplaire du matériel est disponible : `QSG_RENDER_TIMING=1` et une lecture du temps par image.

9. Passer en production

Une liste courte, à vérifier avant d'embarquer :

- ☐ `VehicleSimulator.running` à `false`. Sinon il continuera d'écraser les valeurs venues du bus, en silence.
- ☐ `VehicleData` alimenté par la vraie source — voir [BACKEND.md](#), §« Les trois façons de brancher ».
- ☐ `hvTestPattern` remplacé par le résultat du contrôle réel des cinq organes.
- ☐ Qualité des signaux réellement pilotée : `setQuality(nom, "MISSING")` quand une trame CAN manque, faute de quoi - - ne s'affichera jamais.
- ☐ Points encore en dur passés en revue (liste dans [BACKEND.md](#), §« Points encore en dur ») : navigation, média, téléphone, entretien.
- ☐ Plateforme Qt choisie (`eglfs` en général) et curseur masqué.
- ☐ Fluidité mesurée sur la cible (§8.4).
- ☐ `./test.sh` au vert.

10. Dépannage

Symptôme	Cause	Remède
CMake ne trouve pas Qt 6	Qt hors du système	<code>CMAKE_PREFIX_PATH=\$HOME/Qt/6.8.3/gcc_64 ./run.sh</code>
<code>module "AgoojiyeHMI" is not installed</code>	singleton déclaré après <code>qt_add_qml_module</code>	déplacer les <code>set_source_files_properties</code> avant
Écran noir, aucune erreur	un <code>ShaderEffect</code> a été introduit	le remplacer par du Canvas 2D (§6.6)
<code>spd-say</code> parle, l'application non	plugin de sortie Qt manquant	<code>sudo apt install qt6-speech-speechd-plugin</code> , ou laisser le repli faire son travail
<code>./test.sh</code> échoue en étape 5 sans changement de code	relevé trop court pour voir les 3 phases	<code>HMI_TRACE</code> force déjà le mode déterministe ; vérifier qu'il est bien lu
Argument de <code>run.sh</code> refusé	faute de frappe	<code>./run.sh --help</code> liste les valeurs valides
<code>./run.sh: no such file or directory</code>	mauvais dossier	se placer dans <code>agoojiye-hmi/</code>

11. Pour aller plus loin

- [README.md](#) — démarrage rapide, trois choses à savoir avant de toucher au code
- [BACKEND.md](#) — inventaire complet des propriétés de `VehicleData`, trois méthodes de branchement, points encore en dur
- `./run.sh --help` — tous les scénarios de démonstration
- `qml/VehicleSimulator.qml` — le modèle physique, commenté ligne à ligne